First Responder Bot: Edge-AI Autonomous Navigation & Forensics Rover

Technical Documentation and System Specification Report

Academic Information

- **Course:** 23CSE472 – Artificial Intelligence and Robotics
- **Department:** Department of Computer Science and Engineering
- **Institution:** School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India
- **Project Mentor:** Dr. A. Baskar
- **Student Names and Registration Numbers:**
  - AJAY N MUKUNTH — CB.SC.U4CSE23006
  - ASHWIN V — CB.SC.U4CSE23009
  - SAKTHI KARTHIK B C — CB.SC.U4CSE23010
  - BALAJI N — CB.SC.U4CSE23011
  - KANHA B S — CB.SC.U4CSE23035

1. Executive Summary

The **First Responder Bot** is an edge-computed, autonomous mobile robotic rover developed for air-gapped indoor reconnaissance in

hazardous environments and sensitive crime scene investigations. In forensic investigations, human entry introduces significant risks of physical contamination to fragile biological trace evidence, footwear patterns, and volatile organic compounds. Furthermore, transmitting video streams to off-site cloud servers creates latency bottlenecks and introduces severe digital interception vulnerabilities.

This project delivers an autonomous, 100% offline robotic platform powered by a **Raspberry Pi 5 (8GB)** running an onboard **Ultralytics YOLOv8n deep convolutional neural network**. The perception engine evaluates spatial visual obstruction across three discrete horizontal zones (Left, Center, Right) and applies a vertical Ground Region of Interest (ROI) filter. The mobile chassis uses a 4-wheel drive (4WD) skid-steer architecture actuated by dual L298N H-Bridge motor drivers via hardware-backed Pulse Width Modulation (PWM).

Once local neural network weights and dependencies are installed, the robot requires zero internet connectivity, zero cloud APIs, and zero external computing resources to navigate, recognize obstacles, avoid humans, and safely explore indoor terrain.

---

2. Problem Definition & Operational Context

2.1 Physical Scene Contamination

Forensic investigations are strictly governed by Locard's Exchange Principle: *"Every contact leaves a trace."* When human personnel enter an unsecured crime scene, they risk:

- Inadvertently trampling or altering blood spatter patterns, shell casing positions, and footwear impressions.
- Depositing exogenous biological material (hair, epithelial cells, fibers).
- Agitating dust and aerosolized compounds, distorting environmental forensic signatures.

An autonomous scout rover entering the perimeter prior to human entry provides high-resolution photographic inspection while preserving the physical integrity of the scene.

2.2 Cloud Transmission Vulnerabilities

Traditional teleoperated or cloud-assisted rovers rely on Wi-Fi, cellular, or remote server links. In mission-critical and forensic environments, this introduces significant flaws:

- **Digital Interception & Chain of Custody:** Unencrypted or remote video links risk interception, unauthorized recording, and legal challenges regarding evidentiary chain of custody in court.
- **Signal Attenuation & Blind Spots:** Indoor crime scenes, basements, reinforced concrete facilities, and remote facilities frequently lack reliable RF connectivity.
- **Latency & Teleoperation Instability:** Cloud inference round-trip latencies (>200–500 ms) make real-time collision avoidance unreliable, leading to collisions with unmapped obstacles.

2.3 The Edge-AI Solution

The First Responder Bot executes all visual processing, inference, navigation reasoning, and motor timing directly on the embedded CPU of the Raspberry Pi 5. Operating fully air-gapped guarantees zero digital leakage and predictable real-time reaction cycles under 55 ms.

---

3. System Objectives & Measurable Performance Targets

The engineering implementation is designed to achieve the following performance targets:

1. **Local Edge Vision:** Execute real-time YOLOv8n object detection

locally on the Raspberry Pi 5 CPU without external accelerators, sustaining 15.0 to 18.5 frames per second (FPS) at 320x320 input resolution.
2. **Reflex Action & Collision Braking:** Maintain an emergency halt reaction latency of under 35 ms from perception hazard detection to motor de-energization.
3. **Multi-Zone Spatial Navigation:** Dynamically partition the visual sensor plane into Left, Center, and Right zones with an automated Ground Region of Interest filter to reject ceiling fixtures and background clutter.
4. **Strict Human Safety Protocol:** Detect human presence with dedicated high-sensitivity thresholds, immediately halting if a person is in the central path or veering away if detected in the lateral periphery.
5. **Fail-Safe Watchdog Protection:** Deploy independent background software watchdogs to guarantee motor shutdown if the camera stream drops (800 ms timeout) or if control loop commands stall (500 ms timeout).
6. **Electrical Isolation & Power Integrity:** Maintain complete electrical separation between the high-current 12V inductive motor rail and the sensitive 5V logic rail, utilizing a shared common ground reference to prevent GPIO logic floating.

---

4. Mathematical Modeling & Kinematic Formulations

4.1 Mobile Base Kinematics (4WD Skid-Steer / Differential Drive)

The robot utilizes a 4-wheel drive skid-steer chassis. In planar indoor environments, a 4WD fixed-axle platform is modeled kinematically as an equivalent differential drive system, where rotation is achieved through velocity differentials between the left and right wheel tracks.

Platform Constants:

- Wheel Radius (r) = 0.033 m (33 mm)
- Effective Track Width (B) = 0.160 m (160 mm)
- Forward Cruising Speed Duty = 38%
- Maximum Speed Ceiling Duty = 50%

Governing Kinematic Equations:

Let the global planar position of the robot be defined by coordinates (x, y) with heading orientation angle θ. The forward kinematic relationships mapping left wheel angular velocity (ω_L) and right wheel angular velocity (ω_R) to linear velocity (v) and angular velocity (ω) are:

Linear Velocity: v = (r / 2) * (ω_R + ω_L)

Angular Velocity: ω = (r / B) * (ω_R - ω_L)

Global Position Derivatives: dx/dt = v * cos(θ) = (r / 2) * (ω_R + ω_L) * cos(θ) dy/dt = v * sin(θ) = (r / 2) * (ω_R + ω_L) * sin(θ) dθ/dt = ω = (r / B) * (ω_R - ω_L)

Operational Motion Modes:

- **Forward Cruising (ω_L = ω_R = ω_0):** v = r * ω_0, and dθ/dt = 0 (linear translation without heading change).
- **Pivot Turn Left (ω_L = -ω_0, ω_R = +ω_0):** v = 0, and dθ/dt = (2 * r * ω_0) / B (counter-clockwise rotation in place).
- **Pivot Turn Right (ω_L = +ω_0, ω_R = -ω_0):** v = 0, and dθ/dt = - (2 * r * ω_0) / B (clockwise rotation in place).
- **Full Stop (ω_L = 0, ω_R = 0):** v = 0, and dθ/dt = 0 (complete immobilization).

4.2 Sensor Mast Kinematics (Denavit-Hartenberg Formulation)

For robotic sensor positioning, a 2-Degree-of-Freedom (2-DOF) Pan-Tilt

mast provides analytical mapping between the robot base coordinate system and the optical sensor center.

Joint Architecture:

- **Joint 1 (Pan):** Revolute joint rotating about the vertical Z_0 axis by angle θ1 (azimuth).
- **Joint 2 (Tilt):** Revolute joint rotating about the horizontal Z_1 axis by angle θ2 (elevation).

Denavit-Hartenberg (DH) Parameter Table:

Link (i)	Joint Angle (θi)	Link Offset (di)	Link Length (ai)	Link Twist (αi)
Link 1 (Pan)	θ1	d1 = 0.12 m	a1 = 0.00 m	+90°
Link 2 (Tilt)	θ2	d2 = 0.00 m	a2 = 0.05 m	0°

General DH Transformation Matrix:

The homogeneous transformation matrix T mapping link i to link i-1 is:

```
T(i to i−1) = [
  cos(θi),  −sin(θi)*cos(αi),   sin(θi)*sin(αi),  ai*cos(
  sin(θi),   cos(θi)*cos(αi),  −cos(θi)*sin(αi),  ai*sin(
        0,           sin(αi),           cos(αi),
        0,                 0,                 0,
]
```

Individual Link Transformations:

For Link 1 (α1 = +90°, d1 = 0.12 m, a1 = 0 m):

```
T_1_0 = [
```

```
  cos(θ1),    0,    sin(θ1),       0;
  sin(θ1),    0,   −cos(θ1),       0;
        0,    1,          0,    0.12;
        0,    0,          0,       1
]
```

For Link 2 (α2 = 0°, d2 = 0 m, a2 = 0.05 m):

```
T_2_1 = [
  cos(θ2),  −sin(θ2),    0,    0.05 * cos(θ2);
  sin(θ2),   cos(θ2),    0,    0.05 * sin(θ2);
        0,         0,    1,                 0;
        0,         0,    0,                 1
]
```

Composite Forward Kinematics (T_2_0 = T_1_0 * T_2_1):

Multiplying the transformation matrices gives the composite end-effector matrix:

```
T_2_0 = [
  cos(θ1)*cos(θ2),  −cos(θ1)*sin(θ2),    sin(θ1),    0.05 *
  sin(θ1)*cos(θ2),  −sin(θ1)*sin(θ2),   −cos(θ1),    0.05 *
          sin(θ2),           cos(θ2),          0,    0.12 +
                0,                 0,          0,
]
```

The resulting camera optical center coordinates P = [Px, Py, Pz] relative to the base are:

- Px = 0.05 * cos(θ1) * cos(θ2)
- Py = 0.05 * sin(θ1) * cos(θ2)
- Pz = 0.12 + 0.05 * sin(θ2)

Practical Engineering Implementation:

In dynamic skid-steer locomotion, mobile chassis pitching and sudden

turns introduce mechanical vibration, backlash, and center-of-gravity shifts. To ensure maximum video stream stability and structural durability, the physical robot mounts a wide-angle camera rigidly on the front bumper. This provides a wide, fixed field of view across the forward floor plane while eliminating servo wear, mechanical gear slop, and additional power consumption.

4.3 Spatial Risk Decomposition & Decision Rules

The vision system analyzes incoming frames (Width W = 640 px, Height H = 480 px) by segmenting the visual field into three horizontal columns and a vertical Ground Region of Interest (ROI).

Visual Zone Boundaries:

- **Left Zone:** Normalized X coordinates from 0.00 to 0.35 (35% of frame width).
- **Center Zone:** Normalized X coordinates from 0.35 to 0.65 (30% of frame width).
- **Right Zone:** Normalized X coordinates from 0.65 to 1.00 (35% of frame width).
- **Ground ROI Vertical Threshold:** Normalized Y minimum = 0.35 (top 35% of frame is ignored).

1. Ground ROI Invariant:

An object detection with bounding box coordinates (x1, y1, x2, y2) is evaluated as a physical floor hazard only if its bottom boundary touches the ground region:

is_ground_hazard = True, if (y2 / H) >= 0.35; otherwise False.

This mathematical filter prevents overhead fixtures, ceiling lights, wall hangings, and distant background elements from triggering false obstacle stops.

2. Risk Accumulation Function:

For each lateral zone (Left, Center, Right), the aggregate obstruction risk score R_zone is computed as:

R_zone = min(1.0, Σ [w(class_k) * (Area_k / (W * H))])

Where:

- Area_k = max(0, x2 - x1) * max(0, y2 - y1) is the bounding box area in pixels.
- w(class_k) is the semantic hazard weight assigned to the detected COCO category:
    - person = 1.5 (highest priority)
    - stop sign = 1.0
    - couch, bed, dining table, dog, cat = 0.9
    - chair, potted plant, suitcase = 0.8
    - box, backpack, tv = 0.7
    - laptop, sports ball = 0.6
    - bottle = 0.5
- Obstacles occupying greater than 15% of the total frame area are classified as wide obstacles and contribute an additional 50% risk impact into the Center zone.

3. Deterministic Decision Matrix:

The reactive navigator evaluates perception state and zone risks to issue motion commands:

- **Condition 1 (Camera Failure or Stale Frame):** If camera_ok == False or frame_age > 0.8 seconds -> Action: **STOP** (Fail-Safe emergency halt).
- **Condition 2 (AI Inference Error):** If inference_ok == False -> Action: **STOP** (Perception fail-safe halt).
- **Condition 3 (Human Safety Protocol):** If a person is detected in the center path or occupies >= 25% of the frame -> Action: **STOP**. If a person is detected on the left -> Action: **TURN_RIGHT** (actively veer away). If a person is detected on the right -> Action: **TURN_LEFT** (actively veer away).
- **Condition 4 (Center Path Blocked):** If Center Risk >= 0.12:

    - If Left Risk >= 0.20 and Right Risk >= 0.20 -> Action: **STOP** (All paths blocked / dead end).
    - If Left Risk < Right Risk -> Action: **TURN_LEFT** (Take clear left corridor).
    - If Right Risk <= Left Risk -> Action: **TURN_RIGHT** (Take clear right corridor).
- **Condition 5 (Lateral Buffer Adjustments):** If Left Risk > 0.20 and Left Risk > Right Risk + 0.10 -> Action: **TURN_RIGHT** (Steer away from left obstacle). If Right Risk > 0.20 and Right Risk > Left Risk + 0.10 -> Action: **TURN_LEFT** (Steer away from right obstacle).
- **Condition 6 (Clear Corridor Ahead):** If Center Risk < 0.12 -> Action: **FORWARD** (Forward motion permitted).

---

5. Software Architecture & Implementation Details

The codebase is organized into modular Python modules designed for execution on Raspberry Pi OS Bookworm (64-bit):

5.1 Module Structure

- **`config.py`**: Global configuration file defining GPIO pin mappings, motor speeds, PWM frequency, camera resolution, zone thresholds, and hazard class weights.
- **`camera.py`**: High-performance camera capture module utilizing a background daemon thread (`_capture_worker`). Continuously drains the OpenCV/V4L2 hardware buffer to ensure that calls to `get_frame()` always retrieve the newest captured frame with under 20 ms latency, eliminating stale-frame buffering delays. Includes an automated synthetic mock generator for off-hardware testing.
- **`ai_detector.py`**: Embedded perception engine. Loads the local Ultralytics YOLOv8n model strictly from disk, converts bounding boxes to normalized coordinates, checks the ground ROI

invariant, filters detections by class-specific confidence thresholds, and provides HUD annotation rendering.
- **`navigation.py`**: Reactive spatial navigation engine. Evaluates perception health, computes multi-zone obstruction scores, enforces human safety rules, and generates structured `NavigationDecision` outputs with auditable reasoning strings. Decoupled from hardware for 100% automated unit testing.
- **`motor_controller.py`**: Hardware abstraction layer for dual L298N motor drivers using `gpiozero` and `lgpio`. Implements 4-wire PWM control, software speed capping (50% max duty), direction inversion flags, and an independent daemon thread that enforces an automatic 500 ms safety timeout if control commands cease.
- **`main.py`**: Main application orchestrator. Coordinates startup verification, continuous capture-infer-reason-act loop execution, real-time diagnostic telemetry logging, signal handling (SIGINT/SIGTERM), and graphical HUD display.
- **`download_model.py`**: Automated model preparation utility. Downloads model weights once, computes the SHA-256 integrity hash, and executes an offline CPU warm-up inference to verify air-gapped readiness.
- **`test_motors.py`**: Interactive pre-flight diagnostic tool guiding the user through forward, reverse, pivot left, pivot right, and emergency stop tests with elevated wheels.
- **`diagnose_motors.py`**: Low-level diagnostic utility testing each motor and GPIO pin individually at full 100% and 50% power.
- **`find_right_pins.py`**: Automated pin scanner cycling through candidate Raspberry Pi GPIOs to quickly verify physical wiring.

---

6. Hardware Architecture & Electrical Design

6.1 Bill of Materials (BOM)

Component	Specification	Quantity	Functional Purpose

Single Board Computer	Raspberry Pi 5 (8 GB RAM, Quad-Core Cortex-A76 @ 2.4 GHz)	1	Edge neural network inference, spatial reasoning, and GPIO PWM generation
Vision Sensor	Standard UVC USB Webcam (640x480 RGB @ 15–30 FPS)	1	Forward-facing monocular optical scene capture
Mobile Chassis	4WD Aluminum / Acrylic Chassis with 4 Geared DC Motors	1	Mechanical skid-steer locomotion platform
Motor Drivers	L298N Dual Full H-Bridge Driver Modules	2	Power amplification and dual-track bidirectional motor drive
Logic Power Supply	5V / 3A (15W+) Regulated USB-C Power Bank	1	Dedicated, noise-free power delivery to Raspberry Pi 5
Motor Power Supply	11.1V – 12.6V 3S LiPo Battery Pack	1	Dedicated high-current power for DC motors
Emergency Switch	SPST Heavy-Duty Rocker / Toggle Switch	1	Physical emergency disconnect on the 12V motor power rail

---

6.2 Complete Pin Connection Map

The robot uses two L298N H-Bridge drivers configured in **4-Wire Direct PWM Mode**. The onboard `ENA` and `ENB` headers on both drivers remain shorted with jumpers to the 5V pull-up pins, enabling direct PWM control

on the directional input pins.

Driver #1: Left Side Motors (Front and Rear)

L298N Pin	Connected Function	Raspberry Pi 5 Physical Pin	BCM GPIO Number	Signal Description
ENA	Left Front Speed Enable	—	—	**SHORTED with onboard jumper to 5V**
IN1	Left Front Forward	**Physical Pin 29**	**GPIO 5**	Left Front Forward PWM
IN2	Left Front Reverse	**Physical Pin 31**	**GPIO 6**	Left Front Reverse PWM
IN3	Left Rear Forward	**Physical Pin 36**	**GPIO 16**	Left Rear Forward PWM
IN4	Left Rear Reverse	**Physical Pin 38**	**GPIO 20**	Left Rear Reverse PWM
ENB	Left Rear Speed Enable	—	—	**SHORTED with onboard jumper to 5V**
GND	Ground Reference	**Physical Pin 6**	**GND**	**COMMON GROUND (Tied to Battery (-) & Pi GND)**
12V	Motor Power Positive	—	—	Connected to 12V LiPo Battery (+) via switch

Driver #2: Right Side Motors (Front and Rear)

L298N Pin	Connected Function	Raspberry Pi 5 Physical Pin	BCM GPIO Number	Signal Description
ENA	Right Front Speed Enable	—	—	**SHORTED with onboard jumper to 5V**
IN1	Right Front Forward	**Physical Pin 11**	**GPIO 17**	Right Front Forward PWM
IN2	Right Front Reverse	**Physical Pin 18**	**GPIO 24**	Right Front Reverse PWM
IN3	Right Rear Forward	**Physical Pin 13**	**GPIO 27**	Right Rear Forward PWM
IN4	Right Rear Reverse	**Physical Pin 15**	**GPIO 22**	Right Rear Reverse PWM
ENB	Right Rear Speed Enable	—	—	**SHORTED with onboard jumper to 5V**
GND	Ground Reference	**Physical Pin 14**	**GND**	**COMMON GROUND (Tied to Battery (-) & Pi GND)**
12V	Motor Power Positive	—	—	Connected to 12V LiPo Battery (+) via switch

---

6.3 Electrical Safety & Power Rules

To ensure reliable hardware operation and protect against electrical damage:

1. **Independent Power Supplies:**

    - Power the Raspberry Pi 5 exclusively through its USB-C port using a dedicated 5V / 3A power bank.
    - Power the motors and L298N drivers exclusively from the 12V LiPo battery.
    - Never power the Raspberry Pi directly from unverified step-down buck converters on the 12V battery without verifying exact 5.1V calibration on an oscilloscope or multimeter. Voltage transients from DC motor stalls can cause severe overvoltage and damage the Pi PMIC.
2. **Mandatory Common Ground:**
    - Connect the negative terminal of the 12V LiPo battery, the GND terminals of both L298N drivers, and Pin 6 / Pin 14 (GND) of the Raspberry Pi 5 together. Without a common ground reference, GPIO logic levels float, causing unpredictable motor triggering.
3. **L298N 5V Regulator Jumper:**
    - Keep the onboard `5V-EN` jumper in place on both L298N modules. This allows the internal 78M05 linear regulator to power the L298N logic chip from the 12V motor supply.
    - Leave the 5V screw terminal on the L298N drivers completely unconnected.
4. **Physical Emergency Cutoff:**
    - Always install a physical SPST switch on the 12V positive battery lead within arm's reach during operation.

---

7. Safety Systems & Defensive Engineering

1. **Human Safety Protocol:** People are treated with the highest hazard weighting (1.5). The robot halts if a person is in the center corridor or fills >= 25% of the frame area, and steers away if a person appears in the lateral periphery. The robot never follows or approaches humans.
2. **Camera Stale Frame Fail-Safe:** The camera capture thread records high-precision timestamps for every frame. If the frame age exceeds 0.8 seconds (caused by USB cable disruption or bus lockup), the system immediately halts the robot.

3. **Motor Watchdog Timer:** The motor controller maintains an independent daemon thread that continuously checks the timestamp of the last valid command. If no command is received for 500 ms, all motor outputs are immediately driven to 0.0 duty cycle.
4. **Software Speed Capping:** All motor speeds are constrained by software limits (`MAX_SPEED_DUTY = 0.50`), preventing sudden acceleration indoors.
5. **Emergency Teardown Routines:** Safe exit hooks (`signal.SIGINT`, `signal.SIGTERM`, and spacebar keypress) guarantee that all GPIO devices are closed and motors de-energized during shutdown.

---

8. Verification, Testing & Performance Benchmarks

8.1 Automated Unit Test Suite

The software suite includes 19 deterministic unit tests in the `tests/` directory verifying perception, navigation, and motor control without requiring physical hardware:

```
tests/
├── test_navigation.py       (10 unit tests: corridor cle
├── test_motor_logic.py      (7 unit tests: action dispat
└── test_ai_detector.py      (2 unit tests: Detection str
```

Unit Test Results:

Test Module	Test Case Name

test_navigation	test_clear_corridor_forward

test_navigation	test_center_obstacle_turns_to_clear_left
test_navigation	test_center_obstacle_turns_to_clear_righ
test_navigation	test_all_paths_blocked_stops
test_navigation	test_person_in_center_triggers_human_saf
test_navigation	test_person_on_left_veers_right
test_navigation	test_stale_camera_frame_triggers_fail_sa

test_navigation	test_camera_offline_triggers_fail_safe_s

test_navigation	test_inference_failure_triggers_fail_saf
test_navigation	test_background_ceiling_objects_ignored
test_motor_logic	test_initial_state_is_stopped
test_motor_logic	test_forward_action_updates_state
test_motor_logic	test_reverse_action_updates_state
test_motor_logic	test_pivot_left_action_updates_state

`test_motor_logic`	`test_pivot_right_action_updates_state`
`test_motor_logic`	`test_stop_action_zeros_duty`
`test_motor_logic`	`test_direction_inversion_initialization`
`test_ai_detector`	`test_detection_dataclass_fields`
`test_ai_detector`	`test_mock_detector_interface`

To execute the full test suite:

```
python -m unittest discover tests
```

8.2 Hardware Performance Benchmarks

Physical benchmarks measured during autonomous navigation trials on the Raspberry Pi 5 platform:

Evaluation Metric	Target Specification	Measured Empirical Performance	Operational Assessment
Neural Inference Latency	≤ 35.0 ms	**26.0 ms – 32.0 ms per frame**	Real-time CPU inference achieved without hardware accelerators.
System Loop Throughput	≥ 15.0 FPS	**15.0 – 18.5 FPS (End-to-End)**	Matches webcam capture cadence for responsive reactive control.
Emergency Halt Latency	≤ 50.0 ms	**< 35.0 ms**	Motors brake within a single loop step upon hazard detection.
CPU Utilization	< 80.0%	**38.0% – 46.5% (across 4 cores)**	Ample processing headroom remains; avoids thermal throttling.
Operating Temperature	< 70.0°C	**54.0°C – 61.0°C (active cooler)**	Excellent thermal stability maintained under sustained navigation.

9. Deployment & Operation Guide

9.1 Software Installation on Raspberry Pi 5

Run the following commands on the Raspberry Pi 5 running **Raspberry**

Pi OS Bookworm (64-bit):

```
# 1. Update package lists and install hardware dependenci
sudo apt update
sudo apt install -y python3-pip python3-venv python3-gpio

# 2. Navigate to project root directory
cd /path/to/ai_navigation

# 3. Create virtual environment with system site-packages
python3 -m venv --system-site-packages venv
source venv/bin/activate

# 4. Install required Python packages
pip install --upgrade pip
pip install -r requirements.txt
```

9.2 Offline Air-Gap Verification

Before operating in an air-gapped forensic site, run the offline verification utility:

```
python download_model.py
```

This utility confirms that model weights exist locally in `models/yolov8n.pt`, verifies the SHA-256 hash, runs a test inference on a dummy tensor, and validates that zero runtime downloads or internet requests will occur.

9.3 Motor Calibration (Elevated Wheels)

Elevate the chassis so all four wheels rotate freely in the air, then run:

```
python test_motors.py
```

If a motor channel spins in reverse during the forward motion test:

- Left wheels reverse -> Set `MOTOR_INVERT_LEFT = True` in `config.py`.
- Right wheels reverse -> Set `MOTOR_INVERT_RIGHT = True` in `config.py`.

9.4 Running Autonomous Navigation

Headless Mode (Recommended for SSH / Battery Operation):

```
python main.py --headless
```

The terminal outputs real-time step telemetry:

```
Step 0042 | Action: FORWARD    | Infer: 28.4ms | FPS: 15.
Step 0043 | Action: TURN_LEFT  | Infer: 29.1ms | FPS: 15.
  -> REASON: OBSTACLE AVOIDANCE: Center blocked (0.24), L
```

Visual HUD Window Mode (For Local HDMI Display or VNC):

```
python main.py
```

Displays an OpenCV window rendering bounding boxes, navigation zones (Left/Center/Right), ground ROI divider, and action banners.

- Press **`q`** or **`ESC`** to trigger a graceful shutdown.
- Press **`SPACEBAR`** for immediate manual emergency stop.

---

10. Engineering Limitations

In accordance with scientific and engineering honesty, the following constraints are recognized:

1. **Scale Ambiguity of 2D Vision:** The monocular camera provides

2D pixel coordinates rather than direct metric distance. The robot uses bounding box vertical placement (y2) and area fraction as proximity heuristics. A small object close to the floor can geometrically resemble a larger object further away.
2. **Absence of Wheel Odometry / IMU:** The robot operates on reactive visual feedback. Without wheel encoders or an Inertial Measurement Unit, dead-reckoning position estimation and metric mapping are not supported.
3. **Negative Obstacle Detection:** Standard 2D object detectors classify positive obstacles (chairs, people, boxes). Drop-offs, stairwells, and floor holes are not classified; therefore, operation must be restricted to flat indoor floors.
4. **Transparent & Specular Obstacles:** Clear glass partitions and full-length mirrors are not classified by RGB neural detectors and require future ultrasonic or time-of-flight sensor fusion.

---

11. Future Roadmap

1. **Edge-AI Acceleration via Hailo-8L:** Integrating the Raspberry Pi AI Kit (M.2 HAT+ with Hailo-8L NPU) will increase neural inference throughput to >60 FPS at full 640x640 resolution while reducing CPU utilization to under 10%.
2. **2-DOF Pan-Tilt Mast Implementation:** Fabricating the physical pan-tilt mast with geared digital servos to execute the Denavit-Hartenberg transformation for target-tracking evidence logging.
3. **RGB-D Depth Sensor Integration:** Integrating an Intel RealSense or Luxonis OAK-D depth sensor to obtain true 3D metric point clouds, resolving 2D scale ambiguity.
4. **Visual SLAM & Forensic Mapping:** Deploying RTAB-Map or ORB-SLAM3 for air-gapped 3D metric reconstruction of the crime scene with localized evidence tags.

---

12. References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection.* IEEE CVPR.
2. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8.* GitHub: `https://github.com/ultralytics/ultralytics`.
3. Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control.* Pearson Education India.
4. Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots.* MIT Press.
5. Locard, E. (1930). *The Analysis of Dust Traces.* The American Journal of Police Science, 1(3), 276-298.
6. Raspberry Pi Foundation. (2023). *Raspberry Pi 5 Hardware Architecture and RP1 Peripherals Specification.*